

MASTER PROJECT BUILD PROMPT — EDUPULSE AI

Project Title

EduPulse AI — Intelligent Student Academic Monitoring and Support System

Project Context

Build a complete, functional, software-only web application for a student hackathon project at **KPR Institute of Engineering and Technology (KPRIET)**.

EduPulse AI is an academic monitoring and student-support platform that helps students understand their attendance, marks, academic risk, and recovery options. It helps faculty monitor classes and student progress, and helps administrators manage users, academic settings, and system activity.

The application must be suitable for a live hackathon demonstration. Prioritize a polished, responsive interface, working end-to-end workflows, realistic demo data, and clear user interactions.

Important: Do not build a static UI-only prototype. Implement the frontend, backend, database, authentication, role-based permissions, calculations, notification workflow, and demo/testing mode.

1. Core Objectives

Build a system that can:

1. Monitor student attendance and calculate attendance percentages.
2. Show students how many classes they can safely miss under a configurable attendance requirement.
3. Calculate how many additional classes a student must attend to reach a target percentage.

4. Track subject-wise internal marks, assessment scores, and academic progress.
5. Identify students who may be at academic risk using transparent rules and an optional ML model.
6. Send attendance and marks notifications to real KPRIET student email addresses during an explicitly enabled hackathon test.
7. Allow faculty to enter marks using a voice-assisted input interface, with manual editing as a fallback.
8. Provide separate dashboards for students, professors, and administrators.
9. Provide a test mode that allows the team to demonstrate the system using at least 10 consenting student email recipients.
10. Maintain a clear audit trail for important actions, especially email notification attempts and academic record updates.

2. Technology Stack

Use the following stack unless a dependency is unavailable or a simpler compatible option is required.

Frontend

- React.js with TypeScript
- Vite
- Tailwind CSS
- React Router
- Recharts for dashboards and visualizations
- Lucide React icons
- React Hook Form and Zod for form validation
- Axios or a centralized fetch client for API calls

Backend

- Python FastAPI
- Pydantic for request and response validation
- SQLAlchemy ORM
- PostgreSQL as the primary database
- Alembic for database migrations

- Scikit-learn for an optional academic-risk prediction model

Authentication and Authorization

- Secure password hashing
- JWT-based authentication or secure session-based authentication
- Role-based access control
- Backend authorization checks for every protected endpoint

Email

- Gmail API with OAuth 2.0 authorization for an authorized team member's KPRIET Gmail account
- Use the Gmail `gmail.send` scope where applicable
- Keep OAuth credentials and refresh tokens on the backend only
- Do not store credentials in frontend code or commit them to Git

Development and Deployment

- `.env` configuration
- `.env.example` with placeholder values only
- Docker Compose for local development if practical
- README with setup, configuration, demo login, and testing instructions

Do not expose secrets, OAuth tokens, passwords, or private student information in logs or frontend responses.

3. KPRIET-Inspired Design System

Create a professional, modern institutional dashboard inspired by the visual identity and website style of KPRIET.

Use a blue-and-green institutional palette. Treat the following as proposed UI colors, not verified official brand hex codes.

Suggested Colors

- Institutional blue: #174A8B
- Green accent: #16865B
- Action blue: #2476C7
- Light blue surface: #EAF4FC
- Main background: #F7FAFC
- Card background: #FFFFFF
- Amber warning: #E5A13D
- Red critical: #D64545
- Main text: #172B4D
- Secondary text: #64748B
- Border: #E2E8F0

UI Style

- Clean academic dashboard
- White cards on a light neutral background
- Blue primary buttons and navigation
- Green for positive progress and completed states
- Amber for warnings and attention-needed states
- Red for critical alerts
- Rounded cards with subtle shadows
- Clear typography and sufficient spacing
- Responsive sidebar navigation on desktop
- Mobile-friendly navigation and layouts
- Accessible color contrast and keyboard navigation
- Use charts, progress bars, badges, and status indicators consistently

Avoid excessive gradients, oversized decorative graphics, or cluttered layouts. The application should look like a real institutional product rather than a generic admin template.

4. User Roles and Permissions

Implement three primary roles.

A. Student

Students can:

- View their own attendance by subject.
- View attendance percentage and attendance history.
- Use the bunk-impact calculator.
- Use the attendance recovery calculator.
- View marks and assessment breakdowns.
- View academic-risk status and the reasons behind it.
- View notifications sent to their account.
- View suggested actions for improving attendance or marks.
- Update limited profile information, if enabled.

Students must not be able to edit their own attendance or marks.

B. Professor / Faculty

Professors can:

- View assigned courses and sections.
- View students enrolled in their assigned courses.
- Mark or update attendance for their assigned classes.
- Enter and update marks for assigned courses.
- Use voice-assisted marks entry.
- View attendance and marks summaries.
- View students flagged for attention.
- Trigger an email notification for an eligible student or test recipient.
- View notification status for their authorized courses.

Professors must not be able to modify courses or records outside their assigned permissions.

C. Administrator

Administrators can:

- Manage student, professor, course, and section records.
- Assign professors to courses.
- Enroll students in courses.

- Configure academic thresholds.
- Configure assessment categories and maximum marks.
- View institution-wide academic summaries.
- Manage test recipients and email test settings.
- View audit logs and notification delivery attempts.
- Enable or disable demo/test mode.

Use backend role checks. Hiding a button in the frontend is not sufficient authorization.

5. Authentication and First-Run Experience

Create:

- Login page
- Logout
- Role-based redirection after login
- Unauthorized-access page
- Session-expiration handling
- Demo account support for hackathon presentation

For local development, provide clearly documented demo accounts for:

- Administrator
- Professor
- Student

Use seeded demo records. Do not hard-code production credentials.

The README must explain how to create or reset the demo accounts.

6. Database Design

Create a normalized database with appropriate primary keys, foreign keys, indexes, and timestamps.

Suggested entities:

Users

- id
- full_name
- institutional_email
- password_hash, if password authentication is used
- role: student, professor, admin
- student_id or faculty_id, where applicable
- department
- section
- is_active
- created_at
- updated_at

Institutional email examples should follow the format used by the institution, such as 24cs263@kpriet.ac.in. Treat this as an example format; do not assume every student ID follows the same pattern.

Courses

- id
- course_code
- course_name
- department
- semester
- section
- professor_id
- created_at

Enrollments

- id
- student_id
- course_id
- enrollment_status

AttendanceSessions

- id
- course_id
- session_date
- session_number
- topic, optional
- created_by
- created_at

AttendanceRecords

- id
- session_id
- student_id
- status: present, absent, excused, if institution policy supports it
- updated_by
- updated_at

Ensure duplicate attendance records for the same student and session are prevented.

Assessments

- id
- course_id
- assessment_name
- assessment_type
- max_marks
- assessment_date
- created_by

Marks

- id
- assessment_id
- student_id
- marks_obtained

- remarks, optional
- entered_by
- updated_at

Validate that marks are not negative and do not exceed the assessment maximum.

Notifications

- id
- recipient_user_id, nullable for external test recipients
- recipient_email
- notification_type
- subject
- message
- status: pending, sent, failed
- provider_message_id, if returned
- error_summary, if failed
- created_at
- sent_at

NotificationSettings

- id
- attendance_threshold
- warning_margin
- marks_warning_threshold
- email_enabled
- test_mode_enabled
- updated_by
- updated_at

AuditLogs

- id
- actor_user_id
- action
- entity_type
- entity_id

- timestamp
- safe_metadata

Do not store OAuth access tokens, refresh tokens, or passwords in audit metadata.

7. Student Dashboard

Create a student dashboard with a clear overview.

Summary Cards

- Overall attendance
- Courses below the configured threshold
- Latest marks or overall marks summary
- Current academic-risk status
- Unread notifications

Attendance Overview

Display:

- Subject name
- Classes conducted
- Classes attended
- Attendance percentage
- Required threshold
- Status badge
- Progress bar

Status examples:

- On Track
- Near Threshold
- Below Threshold

Use configurable thresholds. Do not hard-code an assumed KPRIET attendance policy.

Marks Overview

Display:

- Course
- Assessment
- Marks obtained
- Maximum marks
- Percentage
- Latest update date

Academic Support Panel

Show practical, non-alarming guidance, such as:

- “Attend the next 3 classes to reach your target.”
- “Your attendance is close to the configured threshold.”
- “Review the latest assessment results for this course.”

The application should explain why a student is flagged rather than showing only a risk label.

8. Attendance Monitoring and Calculation Engine

Implement attendance calculations in the backend and use the same calculation logic consistently across dashboards, APIs, and notifications.

Let:

- A = number of attended classes
- C = total conducted classes
- T = required attendance percentage, expressed as a decimal

Attendance Percentage

If $C > 0$:

$$\text{attendance_percentage} = (A / C) * 100$$

If no classes have been conducted, display **Not Available** rather than dividing by zero.

Bunk-Impact Calculator

Allow the student to enter a proposed number of future classes to miss, B.

Assuming each future class is conducted and missed:

$$\text{projected_attendance} = A / (C + B) * 100$$

Show:

- Current attendance
- Number of proposed missed classes
- Projected attendance
- Whether the projection remains above the configured threshold
- A clear caution when the projection falls below the threshold

Do not label a student “safe” or “eligible” based only on this calculator. Explain that official eligibility depends on institutional rules and verified records.

Maximum Additional Classes That Can Be Missed

Calculate the maximum integer $B \geq 0$ satisfying:

$$A / (C + B) \geq T$$

Handle edge cases safely:

- If $C = 0$, show that there is insufficient attendance history for a meaningful calculation.
- If current attendance is already below the threshold, show 0 safe additional missed classes.
- Avoid division by zero when $T = 0$.
- If the threshold is invalid, reject it in settings validation.

Attendance Recovery Calculator

Let X be the number of consecutive future classes attended.

$$\text{projected_attendance} = (A + X) / (C + X) * 100$$

Calculate the minimum non-negative integer X needed to reach the target threshold.

If it is mathematically impossible to reach the target under the current assumptions, show an explanation rather than an incorrect number.

Calculator UI

Create two separate tools:

1. **Bunk Impact Calculator**
2. **Attendance Recovery Calculator**

Each calculator must show:

- Inputs
- Formula or brief calculation explanation
- Result
- Visual comparison of current attendance, projected attendance, and target threshold
- Clear warning or success message

Use live calculation when inputs change. Do not require a page reload.

9. Marks Management

Professor Marks Entry

Professors can:

- Select a course
- Select an assessment
- Select a student or section
- Enter marks
- Save marks
- Edit marks where permitted

Validate:

- Marks must be numeric
- Marks must be greater than or equal to zero
- Marks must not exceed maximum marks
- Assessment and course must belong to the professor's assignment
- Prevent accidental duplicate records

Display a confirmation before bulk updates.

Student Marks View

Show:

- Assessment-wise scores
- Total obtained marks
- Total possible marks
- Percentage
- Progress over time, where sufficient data exists

Do not invent grade boundaries. If grades are shown, use configurable institution-approved grading rules or show percentages only.

10. Voice-Assisted Marks Entry

Create a voice-assisted marks-entry feature for professors.

Workflow

1. Professor selects the course and assessment.
2. Professor selects a student or opens a student list.
3. Professor activates the microphone button.
4. The interface listens for a spoken marks value.
5. Recognized text appears in a visible editable field.
6. Professor confirms or edits the recognized value.
7. The system validates the marks against the maximum.
8. Professor clicks Save to persist the marks.

Example voice input:

- “Eighty two”
- “Seventy five”
- “Student 24CS263, marks 82” — only if a reliable student-selection parser is implemented.

Requirements

- Use browser speech recognition where supported.
- Clearly show microphone/listening state.
- Provide manual text entry as a fallback.
- Never save marks automatically from speech without professor confirmation.
- Do not record or store audio by default.
- Show an understandable error if speech recognition is unavailable or permission is denied.

The professor must remain responsible for confirming the recognized marks.

11. Academic Risk Monitoring

Implement an explainable risk-monitoring system.

Initial Rule-Based Risk Engine

Start with a transparent rule-based system so that the project works even without a trained ML model.

Possible configurable indicators:

- Attendance below the configured threshold
- Attendance close to the threshold
- Marks below a configured assessment or course threshold
- Multiple courses with attendance concerns
- Repeated low assessment results, if enough assessment data exists

Use statuses such as:

- Low Concern

- Needs Attention
- High Concern

These are monitoring indicators, not official academic decisions.

Risk Explanation

For each flagged student, show the contributing reasons, such as:

- “Attendance is 3 percentage points below the configured threshold.”
- “Latest assessment score is below the configured marks threshold.”
- “Two courses currently require attention.”

Avoid presenting a risk label without its reasons.

Optional ML Model

If enough appropriate, consented, non-sensitive historical or synthetic data is available:

- Train a simple Scikit-learn model.
- Use only approved academic indicators.
- Split data into training and test sets.
- Report basic evaluation metrics.
- Avoid claiming high accuracy without evaluation.
- Keep the rule-based engine as a fallback.
- Clearly label model output as an estimate, not a definitive prediction.

Do not use sensitive personal attributes or infer personal circumstances.

12. Email Notification System — Real Hackathon Testing

This project will be tested with **at least 10 real KPRIET student email recipients**, using consenting students’ institutional addresses.

The students are recipients. They do not need to provide their Gmail passwords or authorize EduPulse merely to receive email.

Sender Model

Use one authorized team member's KPRIET Gmail account as the sender.

The sender must explicitly complete Google OAuth authorization for the application. Use Gmail API sending permissions, such as the `gmail.send` scope.

Do not:

- Ask students for their Gmail passwords.
- Sign in to student accounts.
- Impersonate a student.
- Spoof sender addresses.
- Put OAuth tokens in frontend code.
- Commit credentials to Git.

If institutional Google Workspace restrictions prevent OAuth access, provide a documented mock-email mode and explain the required administrator approval. Do not bypass institutional restrictions.

Test Recipient Roster

Create an admin-only test-recipient management page.

Allow the administrator to:

- Add a recipient name
- Add an institutional email address
- Mark consent as confirmed
- Enable or disable the recipient for testing
- View test status

Use at least 10 consenting student recipients for the hackathon test.

Use a clear test label in every test message:

[EduPulse Hackathon Test]

Email Types

Support:

1. Attendance approaching threshold
2. Attendance below configured threshold
3. Attendance projection warning
4. Marks updated
5. Academic-risk summary

Example Email

Subject:

[EduPulse Hackathon Test] Attendance Alert – Course ABC

Body:

“Hello [Student Name],

This is a test notification generated by the EduPulse AI hackathon prototype.

Your recorded attendance for [Course Name] is [Attendance Percentage]%.

The configured attendance threshold for this test is [Threshold]%.

Please review your EduPulse dashboard for details.

This message is part of a hackathon test and is not an official examination-eligibility decision.

— EduPulse AI”

Trigger Logic

Notifications should be generated from meaningful events, such as:

- Attendance record saved or updated
- Marks published or updated
- A student crosses a configured warning threshold
- An authorized professor or administrator triggers a test notification

Do not send emails on every dashboard load or browser refresh.

Implement deduplication so the same alert is not repeatedly sent for the same event and recipient.

Email Status Tracking

Store:

- Recipient email
- Notification type
- Subject
- Creation time
- Send attempt time
- Status: `pending`, `sent`, or `failed`
- Provider message ID, if available
- Safe error summary, if failed

A successful Gmail API response means the provider accepted the send request; do not display it as confirmed inbox delivery.

Provide:

- Test email button
- Send-to-one-recipient option
- Controlled batch test for the approved roster
- Confirmation before batch sending
- Clear success/failure summary
- Retry option for failed sends
- Notification history page

Keep email sending on the backend.

13. Professor Dashboard

Create a faculty dashboard with:

Overview Cards

- Assigned courses

- Enrolled students
- Attendance sessions recorded
- Students requiring attention

Course Management

- Course list
- Section information
- Student roster
- Attendance summary
- Marks summary

Attendance Workflow

1. Select course.
2. Select date/session.
3. Load enrolled students.
4. Mark present or absent.
5. Review the summary.
6. Save attendance.
7. Display updated attendance percentages.
8. Optionally trigger a notification for eligible test scenarios.

Marks Workflow

1. Select course.
2. Select assessment.
3. Enter marks manually or with voice assistance.
4. Review and validate.
5. Save.
6. Show confirmation and updated course summary.

Attention List

Show students requiring attention with:

- Student name or ID
- Course

- Attendance percentage
- Marks summary
- Reason for flag
- Last updated date

Limit faculty access to assigned courses and sections.

14. Administrator Dashboard

Create an admin dashboard with:

Summary

- Total students
- Total professors
- Total courses
- Students requiring attention
- Notification send statistics

Management Pages

- Students
- Professors
- Courses
- Sections
- Enrollments
- Academic thresholds
- Assessment settings
- Test recipients
- Email configuration status
- Audit logs

Configuration

Allow authorized administrators to configure:

- Attendance threshold
- Warning margin
- Marks warning threshold
- Email test mode
- Notification enable/disable
- Academic term or semester

All threshold changes must be validated and recorded in the audit log.

15. Analytics and Visualizations

Use Recharts for:

- Attendance by course
- Attendance trend over time
- Marks trend over assessments
- Course-level academic-risk counts
- Notification send outcomes

Charts must:

- Have clear titles and labels
- Include legends where useful
- Use accessible colors
- Show empty states when no data exists
- Avoid misleading axes or unexplained percentages

Provide table alternatives for important chart information.

16. API Structure

Create REST endpoints with consistent response formats and validation.

Suggested endpoint groups:

Authentication

- POST /api/auth/login
- POST /api/auth/logout
- GET /api/auth/me

Students

- GET /api/students/me
- GET /api/students/me/attendance
- GET /api/students/me/marks
- GET /api/students/me/risk
- GET /api/students/me/notifications

Attendance

- GET /api/courses/{course_id}/attendance
- POST /api/courses/{course_id}/attendance/sessions
- POST /api/attendance/sessions/{session_id}/records
- PATCH /api/attendance/records/{record_id}

Calculators

- POST /api/attendance/calculate-bunk-impact
- POST /api/attendance/calculate-recovery

Marks

- GET /api/courses/{course_id}/assessments
- POST /api/courses/{course_id}/assessments
- POST /api/assessments/{assessment_id}/marks
- PATCH /api/marks/{mark_id}

Risk

- GET /api/risk/course/{course_id}

- GET /api/risk/student/{student_id}

Notifications

- POST /api/notifications/test
- POST /api/notifications/batch-test
- GET /api/notifications/history
- POST /api/notifications/{notification_id}/retry

Administration

- CRUD endpoints for users, courses, enrollments, settings, and test recipients.

Enforce role permissions and ownership checks on the backend. Do not rely only on frontend route protection.

17. Seed Data and Hackathon Demo Mode

Provide realistic synthetic seed data for:

- 1 administrator
- 3 professors
- At least 10 students
- 3 or more courses
- Multiple attendance sessions
- Multiple assessments
- Marks for each student
- A mixture of attendance and marks conditions

Use synthetic data for the default demo.

Do not populate real student academic records unless the students and institution have approved that use.

Create a visible demo-mode indicator so evaluators understand which data is synthetic.

The test email recipient list must be separate from synthetic academic seed data. Real recipient emails should only be added by an authorized user with consent.

18. Validation, Errors, and Security

Implement:

- Backend request validation
- Frontend form validation
- Clear loading, success, empty, and error states
- Rate limiting for login and email-sending endpoints where practical
- Authorization checks on all protected operations
- Safe error messages
- Secure password hashing
- Environment-based secrets
- CORS configuration for development and deployment
- Input sanitization where appropriate
- Audit logging for sensitive actions
- No secrets in Git
- No unnecessary exposure of student data

Do not log full email message bodies or OAuth tokens.

19. Testing Requirements

Create tests for:

- Attendance percentage calculation
- Bunk-impact calculation
- Attendance recovery calculation
- Threshold edge cases
- Marks validation
- Duplicate attendance prevention
- Role-based endpoint permissions

- Notification deduplication
- Email mock mode
- Email failure handling

Include manual QA steps for:

- Student login and dashboard
- Professor attendance entry
- Professor marks entry
- Voice marks entry and manual fallback
- Admin threshold configuration
- Test email to one consenting recipient
- Controlled batch email test to the approved 10-student roster
- Notification history and failure handling

20. Required UI Pages

Build these pages with consistent layout and navigation:

1. Login
2. Student Dashboard
3. Student Attendance Details
4. Bunk-Impact Calculator
5. Attendance Recovery Calculator
6. Student Marks
7. Student Academic Risk
8. Student Notifications
9. Professor Dashboard
10. Course Details
11. Attendance Entry
12. Marks Entry
13. Voice-Assisted Marks Entry
14. Faculty Attention List
15. Admin Dashboard
16. User Management
17. Course and Enrollment Management
18. Academic Settings

- 19. Test Recipient Management
- 20. Email Test and Notification History
- 21. Audit Logs
- 22. Unauthorized / Not Found pages

21. Implementation Workflow

Build the application in the following phases. Do not skip directly to a polished frontend while leaving the backend nonfunctional.

Phase 1 — Project Setup

- Create frontend and backend structure.
- Configure environment variables.
- Set up database connection.
- Configure linting and formatting.
- Create README and `.env.example`.

Phase 2 — Database and Authentication

- Create database models and migrations.
- Implement password hashing and authentication.
- Implement role-based access control.
- Seed demo users and synthetic data.
- Verify that each role sees only permitted data.

Phase 3 — Core Academic Workflows

- Implement courses and enrollments.
- Implement attendance sessions and records.
- Implement marks and assessments.
- Add validation and audit logging.

Phase 4 — Student Features

- Build student dashboard.
- Build attendance and marks views.
- Implement bunk-impact calculator.
- Implement recovery calculator.
- Build academic-risk explanations.

Phase 5 — Professor Features

- Build course and roster views.
- Implement attendance entry.
- Implement marks entry.
- Implement voice-assisted entry with confirmation and manual fallback.

Phase 6 — Admin Features

- Build user/course management.
- Add academic threshold settings.
- Add test recipient management.
- Add audit log views.

Phase 7 — Notification System

- Implement mock email mode first.
- Add Gmail OAuth configuration on the backend.
- Implement single-recipient test sending.
- Add consent-confirmed test roster.
- Add controlled batch testing.
- Add deduplication, status tracking, and retries.
- Verify email sending with one authorized test recipient before testing the full roster.

Phase 8 — Analytics and UI Polish

- Add charts and progress indicators.
- Apply KPRIET-inspired palette.
- Improve responsive behavior.

- Add empty, loading, and error states.
- Ensure all navigation and buttons work.

Phase 9 — Testing and Final Demo

- Run backend and frontend tests.
- Verify role restrictions.
- Verify calculations with edge cases.
- Verify email mock mode.
- Verify real email sending only after OAuth authorization and recipient consent.
- Prepare a demo walkthrough.
- Document limitations and setup instructions.

22. Expected Final Deliverables

Produce:

1. Complete frontend source code.
2. Complete backend source code.
3. Database models and migrations.
4. Seed-data script.
5. `.env.example`.
6. README with installation and run instructions.
7. Demo login details for local synthetic-data testing.
8. Gmail OAuth setup instructions.
9. Test-email instructions.
10. API documentation through FastAPI OpenAPI/Swagger.
11. Automated tests for key calculations and permissions.
12. A short hackathon demo script showing the student, professor, and administrator workflows.

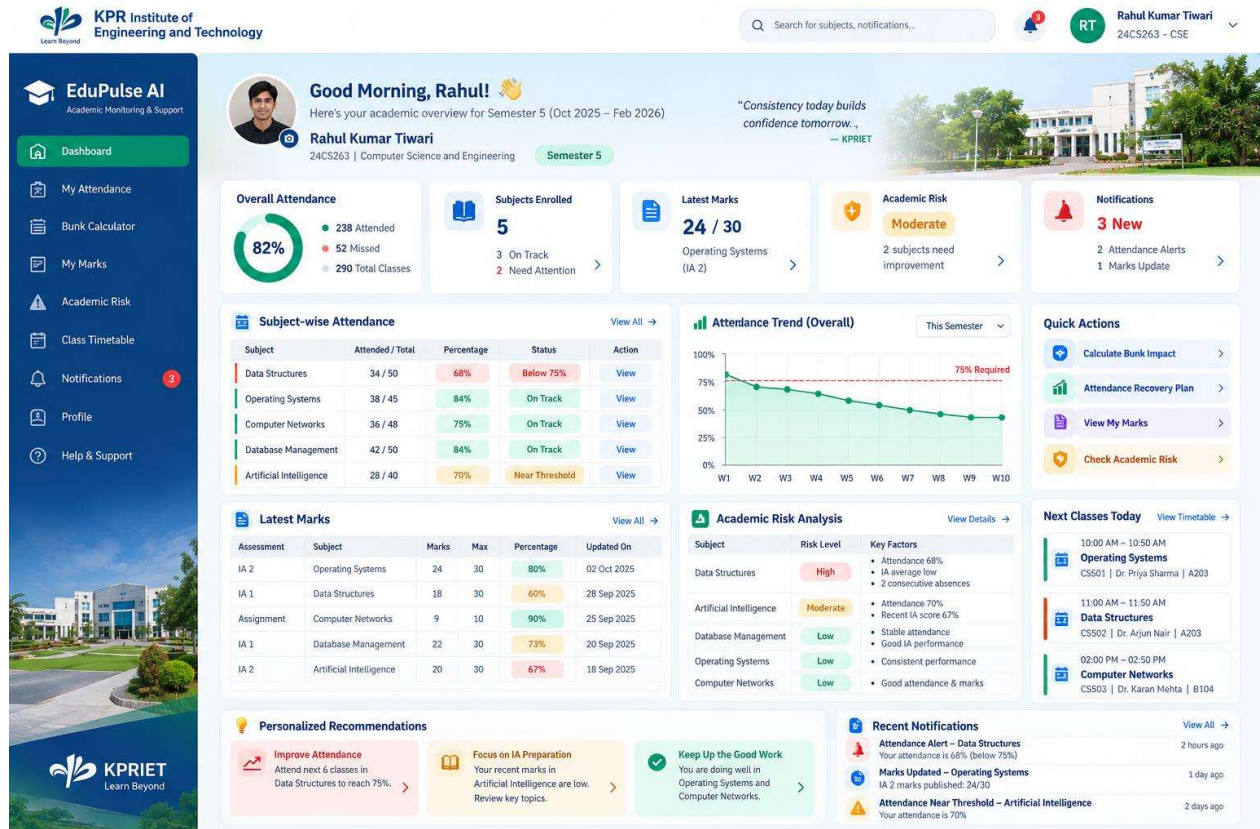
23. Development Instructions for the AI Coding Agent

- First inspect the current project directory and existing files.
- If the repository is empty, initialize the project structure.
- If code already exists, preserve working functionality and extend it rather than replacing it unnecessarily.
- Implement the application incrementally, phase by phase.
- Do not leave buttons as decorative placeholders.
- Do not use hard-coded frontend arrays as the permanent data source.
- Connect the frontend to the backend API.
- Use synthetic seed data for demonstrations.
- Keep the email feature disabled or in mock mode until OAuth credentials are configured.
- Never invent an existing KPRIET sender mailbox. Use the authorized team member's actual account after consent and OAuth authorization.
- Do not assume an official attendance threshold or grading policy. Make these configurable.
- After each phase, run the relevant checks and fix errors before proceeding.
- Provide a final summary of implemented features, how to run the app, and any configuration still required.

Primary goal: Deliver a working, secure, responsive academic monitoring prototype that can be demonstrated end-to-end during the hackathon, including real test email delivery to at least 10 consenting KPRIET student recipients after the sender account is authorized.

Sample UI screens

1. Student Dashboard



2. Professor Dashboard

**KPR Institute of Engineering and Technology**
Learn Beyond

**EduPulse AI**
Academic Monitoring & Support

Dashboard

My Courses

Attendance Entry

Marks Entry

Voice Mark Entry

Student Analytics

At-Risk Students

Notifications 5

Reports

Profile

Help & Support



**KPRIET**
Learn Beyond



Good Morning, Dr. Priya Sharma!
Here's the overview of your academic activities.
Assistant Professor | Department of Computer Science and Engineering

 Search students, courses, or notifications...

 **Dr. Priya Sharma**
Assistant Professor - CSE

Academic Year 2025 - 2026 | Odd Semester

**Assigned Courses**
4
3 Theory | 1 Lab

**Total Students**
186
Across all courses

**Today's Classes**
2
1 Completed | 1 Upcoming

**Students Need Attention**
28
15 Low Attendance | 13 Low Marks

**Pending Tasks**
4
2 Attendance | 2 Marks

My Courses (Current Semester)

View All →

Code	Course Name	Semester	Section	Students	Actions
CS501	Data Structures	III	A	52	View
CS502	Operating Systems	III	A	48	View
CS503	Computer Networks	III	B	46	View
CS504L	Database Lab	III	A	40	View

Attendance Overview (My Courses)

This Week



Course	Present	Absent	Attendance %
CS501	40	12	78%
CS502	35	13	82%
CS503	32	14	74%
CS504L	38	2	88%

Quick Actions

[Mark Attendance](#)

[Enter Marks](#)

[Use Voice Mark Entry](#)

[View Student Performance](#)

Today's Schedule

View Timetable →

 10:00 AM - 10:30 AM
Data Structures (CS501)
Theory Class | A203
[Completed](#)

 11:00 AM - 11:50 AM
Operating Systems (CS502)
Theory Class | A204
[Take Attendance](#)

 01:00 PM - 01:30 PM
Faculty Meeting
Conference Hall
[Upcoming](#)

 02:00 PM - 02:30 PM
Student Mentoring
Department Room
[Upcoming](#)

Students Requiring Attention

View All →

Roll No	Name	Course	Attendance	Marks	Risk	Action
23CS101	Rahul Tiwari	CS501	68%	55%	High	View
23CS104	Sneha Verma	CS502	72%	62%	High	View
23CS118	Arjun Nair	CS503	70%	48%	High	View
23CS121	Kavya S	CS501	76%	58%	Moderate	View
23CS135	Vikram R	CS504L	82%	64%	Moderate	View

Recent Notifications

View All →

 **Attendance Alert - CS501**
23CS101 (Rahul Tiwari) is at 68%
2 hours ago

 **Marks Published - CS502**
IA 2 marks published for 48 students
1 day ago

 **Student Risk Update**
3 students flagged as High Risk
1 day ago

 **Attendance Reminder - CS503**
Next class tomorrow at 10:00 AM
2 days ago

 **Voice Entry Successful**
Marks entered for CS501 (12 students)
3 days ago

Marks Entry (Manual)

Open Full Page →

Course: CS501 - Data Structures

Assessment: IA 2 (Max: 30)

[Save All](#)

Roll No	Student Name	Marks (/ 30)	Status
23CS101	Rahul Tiwari	24	Saved
23CS102	Aditi Sharma	26	Saved
23CS103	Karan Mehta	18	Saved

Voice-Assisted Mark Entry

How to use?



Click and speak the marks

Example: "Enter 24 marks for Rahul Tiwari, IA 2, out of 30"

☒ Auto-detect student from voice

Class-wise Attendance (CS502 - Operating Systems)

View Details →

Date	Session No	Topic	Present	Absent	Attendance
15 Sep 2025	1	Introduction	42	6	88%
17 Sep 2025	2	Process Management	40	8	83%
19 Sep 2025	3	CPU Scheduling	38	10	79%
22 Sep 2025	4	Memory Management	36	12	75%
24 Sep 2025	5	File Systems	39	9	81%

3. Admin Dashboard



4. Home



KPR Institute of
Engineering and Technology

Learn Beyond

Home

About

Features

Roles

Contact

Search

Login

Get Started

EduPulse AI

Track. Predict. Alert. Support.

An Intelligent Academic Monitoring and Support System for KPR Institute of Engineering and Technology.

Stay informed about your attendance, marks, and academic progress with AI-powered insights and timely notifications. Build better learning habits and a brighter future.

Login with KPRIET Email

Explore Features

Attendance Monitoring

Marks & Assessments

AI Risk Analysis

Voice-assisted Mark Entry

Email Notifications

Role-based Dashboards

1,200+ Students

80+ Faculty Members

130+ Courses

6 Departments

Student Portal

Track your attendance, view marks, get personalized insights and receive timely notifications.

Student Login

View Attendance

Check Marks

Bunk Calculator

Academic Risk

Email Alerts

Professor Portal

Manage attendance, enter marks (with voice support) and monitor student performance.

Faculty Login

Mark Attendance

Enter Marks

Voice Entry

View Student Analytics

Identify At-Risk Students

Administrator Portal

Manage students, faculty, courses, configure academic settings and view institution-wide analytics.

Admin Login

User Management

Course & Enrollments

Academic Analytics

Notification Settings

Reports & Export

How EduPulse AI Works

1. Login

Access using your KPRIET email account

2. Academic Data

Manage attendance, marks and courses

3. Smart Analysis

Predict trends and identify risks

4. Automatically Notify

Send email alerts for attendance and marks

5. Take Action

Follow insights and improve performance

Stay Ahead

Get notified about attendance shortage, marks updates and academic insights.

"Education is not just a destination, but a journey we support."

KPRIET